

Northern Illinois University

XML: An emerging standard for data exchange

A Thesis Submitted to the

University Honors Program

In Partial Fulfillment of the

Requirements of the Baccalaureate Degree

With University Honors

Department of Computer Science

By Kevin F Sweeney

b  t

DeKalb, Illinois

May 8, 1999

HONORS THESIS ABSTRACT
THESIS SUBMISSION FORM

AUTHOR: Kevin F Sweeney, [REDACTED]

THESIS TITLE: XML: An emerging standard for data exchange.

ADVISOR: Kurt McMahon ADVISOR'S DEPT: Computer Science

DISCIPLINE: CSCI/Applied YEAR: 1999

PAGE LENGTH: 32 BIBLIOGRAPHY: Yes ILLUSTRATED: No

PUBLISHED: NO

COPIES AVAILABLE: Hard Copy, contact author via email for further availability

ABSTRACT:

The World Wide Web is ever changing. New standards are continuously emerging. One of the newest technologies to emerge is called XML or eXtensible Markup Language. XML is so exciting because it is extensible, meaning that one can define the tags that one wishes to use instead of having to rely on a predefined set of tags as with HTML. In the very near future it appears that HTML will be redesigned in XML to create a new standard called XHTML. With XML, it is possible to describe data in whatever way is needed. This has major implications for communication on the World Wide Web. It is now possible to create tags that accurately describe the data they contain.

For the most part the material for this document was found on the various web sites listed in the resource section, although a very good introduction was gained by reading the book listed there. There is plentiful information about XML on the World Wide Web Consortium's page at <http://www.w3.org>.

XML: An emerging standard for data exchange

by

Kevin F Sweeney



CSCI 499

Kurt McMahon

21 April 1999

Introduction

The World Wide Web is ever changing. New standards are continuously emerging. When considering a topic for my Honors Capstone, I knew that I wanted to pick such an emerging standard to research, because new Web technologies can be very exciting. I picked a technology called XML. I set out to learn about XML, gather information about its uses, and gather information about XML resources. Initially I thought that my Capstone project might also include a program or two that would demonstrate the capabilities of XML. What I soon learned is that because XML is still an emerging standard, and because XML is an extensible language, writing such a program would be a task too great for me to undertake. While my classes at NIU have well prepared me for the world of data processing, I've had very little experience with the programming techniques required to create an XML parser. The programs I could create would work only with my specific XML document, and no other. Obviously such a program is useless when considering the fact that the purpose of XML is that it be extensible and universal. So, my objectives have changed somewhat in the course of my research. I've focused much more on the subject of researching XML and its uses, as well as collecting information about XML resources. This itself is a valuable goal, as anyone who spends much time searching for information on the Internet already understands. Therefore, my project will consist of three parts. The first is a document that describes XML and its uses. The second is a tutorial on how to create an XML document. This should give the reader a good idea of what an XML document might look like. The third is a resource section, where

the reader can look for further information about XML technology. This resource section will also serve as my bibliography. My goal with this project is to introduce the reader to XML and its uses. I hope that the reader will find that I have accomplished this goal.

Section 1: About XML

XML or eXtensible Markup Language is a subset of SGML (Standardized Generalized Markup Language). SGML is by no means a new standard for document markup; it became an international standard in 1986. However, XML is a relatively new standard, and is still an emerging technology. What exactly is XML? XML is a markup language that has great potential for the World Wide Web and for data exchange in general. As its name implies, XML is eXtensible, meaning that it is changeable, and the constructs of the language are not set in stone. For instance in HTML (HyperText Markup Language) there is a certain set of tags or descriptors that are available to control the appearance of a document, and we must be content with what tags are defined as part of the standard. With XML, it is possible to create tags that can describe data in whatever way is needed. This has major implications for communication on the World Wide Web and in data communications in general. It is now possible to create tags that can accurately describe data. This does not only apply to the way that data is displayed on a screen or on paper.

What is a markup language?

A markup language is simply a method of describing data. The printing industry has used markup languages for many years, although they may not have been known by that name. HTML is a markup language. PostScript is a markup language. TeX and LaTeX are markup languages widely used within the Unix community. Each word processor has its own specific markup language. In the cases mentioned, the markup languages are limited to describing data so that it can be output to either a printer or a monitor in a format readable by

humans. A markup language allows data to be described in a format that is readable to the user of the data. The user of the data is generally assumed to be a human, and hence the data is displayed in a format that is human-readable. XML will soon change this assumption.

How is XML different from HTML?

HTML is the standard for World Wide Web markup. People use HTML to create web pages, and browsers use HTML to display web pages. Contained within each browser and HTML editor is something called a DTD (Document Type Definition). A DTD is what tells the browser how to interpret the data it receives, and what tells the HTML editor how to create an HTML document. The user of a browser or an HTML editor never sees the DTD, because it is embedded within the application program. If you'd like to see the DTD for HTML, you can find it on the World Wide Web at <http://www.w3.org/TR/REC-html40/loose.dtd>. With XML, the person who creates the XML document designs the DTD. The DTD is either a part of the XML document or the XML document is linked to the DTD.

To better understand what a DTD does, it is good to understand how a web browser displays a page composed of HTML. First, the user puts in a request to view a web page. Second, the browser uses HTTP (HyperText Transfer Protocol) to retrieve the document from its storage location on the server computer, and places it in a temporary location or a 'cache' on the client computer. The browser then begins to parse the HTML document. Parsing means that the browser breaks the HTML document down into its parts as

defined by the HTML tags contained in the document. The browser knows that the document is in HTML format because the document explicitly states that it is in HTML format with the <HTML> tag which appears at the beginning of every HTML document. The browser then parses the document by finding all of the HTML tags contained within, and using those tags to display the document on the user's screen. The user can then understand the information presented.

The above description does take some liberties with the format that HTML pages are actually composed in. For instance, most browsers are intelligent enough to correct common mistakes in the composition of HTML documents. A common mistake is the use of the <P> or paragraph tag without the use of the closing </P> or end-of-paragraph tag. Since HTML is designed to be easy to use and understand, browsers have evolved to be able to correct for common mistakes transparently. This means that the browser corrects for the mistake without notifying the user that there is anything wrong with the HTML document.

In most cases, we assume that the 'user' of an HTML document is a human user. XML does not presuppose that only human users will view the document it describes. XML is designed to also be viewed by non-human users or data agents. One of the most exciting parts of XML is that fact that it can be easily viewed and understood by non-human users. This brings us to our next topic.

What is the format of XML?

The exact format of the current version of XML or XML 1.0 is described in detail on the W3 or World Wide Web Consortium's web site. You can find the

proposal at <http://www.w3.org/XML/>. An in-depth description of an XML is beyond the scope of this document, and would take tens of pages to describe, so it would be best to look to the above web page in order to find such a description.

A basic XML document contains three parts: a DTD, data, and a style sheet. The best way to understand an XML document is to see one. The following is a hypothetical XML document for a member data document. Each record in the data document would be in the following format:

```
<member>
  <name>
    <first>Kevin</first>
    <last>Sweeney</last>
  </name>
  <address>
    <street>PO BOX [REDACTED]</street>
    <city>DeKalb</city>
    <state>Illinois</state>
    <zipcode>60115-[REDACTED]</zipcode>
  </address>
  <phone>
    <areacode>815</areacode>
    <localnum>7[REDACTED]9</localnum>
  </phone>
  <email>h[REDACTED]u</email>
  <status>active</status>
</member>
```

As you can see, the tags in XML are extensible, and they are defined by the creator of the XML document through the DTD. An XML document can be shaped for a specific purpose. For instance, I need not have separated the name into its component parts, but I chose to do so in this instance. Designing an XML document requires many of the same skills as designing a database. In order to create a useful XML document it is necessary to thoroughly consider what use the data will be put to, and describe the document accordingly. In fact,

XML documents can be created from databases simply by assigning a tag to each needed field in the database, and writing a program that accesses the database to copy the data to an XML document. In looking at an XML document it becomes evident that XML probably will not be useful as a data storage medium because of the amount of space that the tags would consume. However, it is a simple task to write a program that can copy an XML document into a database or copy a database to an XML document.

To display XML data to a computer screen, an XML document can be linked with a style sheet such as XSL (eXtensible Stylesheet Language) or CSS (Cascading Style Sheet) document. The instructions for linking an XML document so a Style Sheet document can be found within the XML proposal at the W3's web site. A style sheet document tells an XML viewer how to display the data marked up by a tag. A style sheet document, for instance, might contain information about what font, font size, and character properties to assign to a certain type of data within an XML document. The style sheet could define that that data within the <email> tag is to be displayed in 12pt, bold font. This is much the same as the tags that currently describe HTML data. However, with XML it is possible to assign such a description to a class of XML data types rather than having to repeatedly describe data.

What are some possible uses for XML?

The emergence of XML brings about exciting possibilities for data exchange. XML is a format that can be used by both humans and computers. Motorola currently has a technology called VoXML that would allow for a call-

answering center to be totally automated. For instance, a user could call a company's technical support phone center, and a computer would answer the phone. The computer would ask the user what type of problem that he or she is having, and would use speech-to-text translation software to convert the user's response to text. The computer could then search the user's response for key words, look up those key words in a database, and then read the information it found back to the user. The same computer could be used to run a technical support web site for the company. A user could go to the web site and enter the type of problem that he or she is having, and the computer could search the text entered for key words, and return the data as described above. XML can be used to format the data so that it can more easily be used in both a web-based and voice-based application.

XML would also increase the capability of users to search the World Wide Web more effectively. Since XML is better able to accurately describe data than HTML, a search agent could better categorize the data found on a web page. Any frequent user of the web knows that in a given search, about 70% of the data that is returned by a search engine is not useful. XML could possibly increase the effectiveness of search engines so that 90% of the data returned to a user is useful.

XML allows for better data exchange between corporations. Currently, if a company is to exchange data with another, it must find out what format the company uses to describe its data, and convert to that format. XML allows for a company to output data in a common format. The company on the other side of

the transaction would have a very simple time converting the data into a format it could use. A current example of this type of data exchange is the OFX or Open Financial eXchange format used by certain financial corporations on the internet to allow for secure financial transactions to take place over the web.

CDF or Channel Definition Format is another example of how XML technology is currently in use on the World Wide Web. CDF allows web site maintainers to let web page viewers know when the content of a page changes. Users can select the type of change they would like to be notified about, and the web server automatically informs the users when such a change has taken place. CML or Chemical Markup Language is a format that allows information about chemicals to be stored in a common format that can be reused. By storing the data in a common format, this standard allows for users to examine only the data that they are interested in.

What impact will XML have on data exchange?

The emergence of XML has widespread significance, specifically on the development of the World Wide Web. XML can increase the effectiveness of data stored on the World Wide Web, and at the same time decrease the amount of bandwidth that it takes to view a document. Because XML allows a user to specify what type of information they would like, and only retrieve that information, it is possible for a user to view only the portion of the document that concerns them, rather than having to download the entire page and search through the document. Also, since XML can be linked to a style sheet, it is no longer necessary to apply style tags to all of the documents on a particular web

site. All documents on a web site that use a common format can simply be linked to a common style sheet. A web browser only need download this style sheet once in order to display all of the pages on a site. Also, a web site designer need only make a change in the style sheet document in order for the change to be reflected across the entire web site.

Non-human data agents can more easily understand a document if the style tags are not contained within the document. Since an XML document need not contain information about how a document is displayed, web search agents can more effectively focus on the data rather than the formatting. Companies can release data in a format that can be used by many more than just one vendor. XML also limits the amount of redundant data that must be stored. For instance, currently it may be necessary to store data in one format for human use and several other formats for use by computers. XML makes this redundancy unnecessary by allowing the reuse of the same data by all those who to view it.

XML is an emerging standard on the web, and will have widespread impact on the way that data is described. Currently both Microsoft and Netscape have plans to implement XML capabilities in their version 5.0 browsers. Microsoft's support for XML is somewhat stronger, and more information about it can be found at <http://www.Microsoft.com/XML>. Information about Netscape's progress can be found at <http://home.netscape.com/browsers/>. With the recent release of Microsoft's Internet Explorer 5.0 beta versions, it is now possible to see how XML works in an interactive format.

Very recently a group of W3 members have put forth a proposal that HTML be redefined in XML, to create a new markup language called XHTML (eXtensible HyperText Markup Language). This standard would make HTML documents much more useable, and would bring XML to the forefront of World Wide Web technology. More information about this proposal is available on the W3's HTML homepage. XML offers exciting possibilities for the future of the World Wide Web and other types of data exchange. While one can make an educated guess about the impact XML will have, at this early point in its development the future of the language is uncertain. XML is certainly a language to watch!

Section 2: Creating an XML document

Possibly the first question that many will ask when they hear about XML is “What does an XML document look like?” In reality, an XML document has several parts, and is usually composed of more than one file. The XML document also changes shapes depending on how it is used. Here I will give a brief tutorial about how one might create an XML document that can serve several purposes. I will throughout this tutorial, use the example of an amateur radio club member database, as I am currently the secretary of such a club, and therefor well acquainted with the type of data that needs to be maintained. The type of data to be maintained will of course be different for your purposes, as shown in the first step. Please remember that this is a **very** basic tutorial. A thorough understanding of all of the possibilities implicit in XML would fill many volumes. This tutorial is only intended to give a very brief synopsis of what an XML document might look like.

Step 1: Decide what data needs to be stored.

This step is not particular to XML. In any case where a type of database is being designed, it is first important to decide exactly what type of data will be stored. The nice thing about XML is that it's not necessary to decide exactly what the definition of each field will be at this point. Since an XML document can be used by many different agents, all data is stored as text, and is converted from text to the appropriate format. In my example, a database of club members should contain quite a few fields. First off, I must declare a 'root element'. The root element is an element that cannot be contained within any other element. I

will call my root element 'memberdata'. With that in mind, here is the first step to my XML document:

```
<memberdata></memberdata>
```

As one can see, the format for creating an XML tag is much like the format used in HTML for creating tags. The data for the membership database is contained between the opening tag <memberdata> and the closing tag </memberdata>.

Within the member database, the individual member data needs to have a place to be stored. I will call this element 'member'. Here is the second step to the XML document:

```
<memberdata>  
    <member></member>  
</memberdata>
```

I will take this opportunity to lay out one of the most important rules regarding XML documents. Element tags **may not overlap**. To illustrate:

```
<memberdata>  
    <member>  
    </memberdata>  
</member>
```

This document would cause a world of difficulties for an XML parser. In some cases, the final </member> tag would simply be ignored, as the root element closes before it is found. In this case an error would be created because the 'member' element did not have a closing tag. In other cases, the </memberdata> element would be ignored because the parser has not yet found a closing tag for the 'member' element. The type of error that would occur depends on the implementation of the particular parser.

Now that I have defined the very basic structure of the document, I need to know what type of data I should keep about each member. For my purposes, a member record should contain a callsign, name, address, phone number and license class of the members. An amateur radio callsign is exactly like the callsign that would be assigned to any local radio station. Each callsign uniquely identifies a single amateur radio operator. I also need to know the email address of each member, if they have one. I need to know whether the member belongs to the ARRL (Amateur Radio Relay League). Since my club is an ARRL-affiliated club, we must at all times make sure that at least half of our members are also members of the ARRL. Our membership year runs from October to September, regardless of the member, so I don't need to keep any information about membership dates. I should also include a comments field for miscellaneous data about the member. Here is how the XML document will look at this point:

```
<memberdata>
  <member>
    <callsign></callsign>
    <name></name>
    <address></address>
    <phone></phone>
    <class></class>
    <email></email>
    <arrl_member></arrl_member>
    <comment></comment>
  </member>
</memberdata>
```

Notice that indentation is important in an XML document just as it is important in any programming language source code file. The indentation is not important to the machine, but it is important to a human reader who is attempting to look at an XML document in its basic form. This is important to note because

one of the benefits of XML is that the data is immediately readable to a human user.

The name element should probably be broken down. While breaking down fields I will only deal with the individual element for the sake of simplicity, and then I will give a complete view of the document when I'm finished. I would like to have a prefix, first name, middle initial, last name, and suffix for each member name. The resulting changes are:

```
<name>
  <prefix></prefix>
  <fname></fname>
  <minit></minit>
  <lname></lname>
  <suffix></suffix>
</name>
```

The address element also must be broken down. I would like to have a street address, a city, a state, and a zipcode for each address. The resulting changes are:

```
<address>
  <street></street>
  <city></city>
  <state></state>
  <zipcode></zipcode>
</address>
```

I also would like to break the phone element down into area code and local phone number.

```
<phone>
  <areacode></areacode>
  <localnum></localnum>
</phone>
```

One last important point is that an XML document should start with an XML declaration. Just as in HTML, it is helpful to have the first line of a file tell us

that this is indeed an XML document, and what version of XML it is written in.

HTML has several versions, and a browser looks at the HTML declaration to better decide how to deal with a document. Although not strictly required, a declaration is important. Since the current version of XML is 1.0, the first line of the file should be:

```
<?xml version="1.0"?>
```

Another line I'll add to the document is the statement that will link the XML document to its DTD. More about this in the next step.

```
<!DOCTYPE memberdata SYSTEM "sample.dtd">
```

The resulting document, with all changes in place, looks like this:

```
<?xml version="1.0"?>
<! DOCTYPE memberdata SYSTEM "sample.dtd">
<memberdata>
  <member>
    <callsign></callsign>
    <name>
      <prefix></prefix>
      <fname></fname>
      <minit></minit>
      <lname></lname>
      <suffix></suffix>
    </name>
    <address>
      <street></street>
      <city></city>
      <state></state>
      <zipcode></zipcode>
    </address>
    <phone>
      <areacode></areacode>
      <localnum></localnum>
    </phone>
    <class></class>
    <email></email>
    <arrrl_member></arrrl_member>
    <comment></comment>
  </member>
</memberdata>
```

This file would be saved as 'sample.xml'. In a review of the document, it appears that I have a space for all the data that I need. It is easier to add elements to an XML document after it has been created than to add fields to a traditional database, but it's still important to make sure you've got all the data you'll need right up front so that a minimum of changes are necessary later. In case you're not sure, there would be more than one 'member' element in the file, unless of course there were only one member in the club.

Step 2: The DTD.

Strictly speaking, an XML document does not need a DTD (Document Type Definition) to be complete. However, a DTD is necessary if we are to parse the document, because it tells an XML parser what format the document should be in. For some reason, DTDs do not seem to be indented as one would think. A DTD is very much like an XML document , for obvious reasons. Here is the basic DTD for my XML document:

```
<?xml version="1.0"?>
<!DOCTYPE memberdata [
]>
```

As you can see, a DTD document contains an XML declaration just like an XML document. The DOCTYPE declaration simply declares what type of document this is. The above is not a useful DTD, as it is necessary to add more information for a parser to be able to check the associated XML document. All that the parser could determine from the DTD as stated is that a document called memberdata exists. Here's the updated version:

```
<?xml version="1.0"?>
<!DOCTYPE memberdata [
<!ELEMENT memberdata (member)+>
]>
```

Even this version doesn't tell the parser much. Now the parser knows that a document called 'memberdata' exists, and it has a root element of 'memberdata.' The parenthesis () around the 'member' element indicate that the element group. The plus + after the 'member' element indicates that the group is repeating, and that there must be a minimum of one element.

The common symbols, with their meanings are:

'	Strict Ordering
	Selection
+	Repetition (1 to many)
*	Repetition (0 to many)
?	Optional element
()	Grouping

Here's another update:

```
<?xml version="1.0"?>
<!DOCTYPE memberdata [
<!ELEMENT memberdata (member)+>
<!ELEMENT member (callsign, name, address, phone+,
                    class, email+?, arrl_member,
                    comment?)>
]>
```

The new line defines the 'member' element. The parser now knows that the 'member' element must contain elements called callsign, name, address, phone, class, email, arrl_member, and comment. The parser knows that the 'phone' element could have more than one occurrence. The parser knows that the email element could have more than one occurrence, and that it is optional because of the question mark after the element. The parser also knows that the 'comment' element is optional. Unless otherwise specified, all elements of an

XML document are required. Depending on the parser, elements may or may not need to occur in the order listed in the DTD. A further update:

```
<?xml version="1.0"?>
<!DOCTYPE memberdata [
  <!ELEMENT memberdata (member)+>
  <!ELEMENT member (callsign, name, address, phone+,
                    class, email+?, arrl_member,
                    comment?)>
  <!ELEMENT callsign (#PCDATA)>
  <!ELEMENT name (prefix?, fname, minit?, lname,
                 suffix?)>
  <!ELEMENT address (street+, city, state, zipcode)>
  <!ELEMENT phone (areacode, localnum)>
  <!ELEMENT class (#PCDATA)>
  <!ELEMENT email (#PCDATA)>
  <!ELEMENT arrl_member (#PCDATA)>
  <!ELEMENT comment (#PCDATA)>
]>
```

This is a fairly complete DTD. The #PCDATA declaration tells the parser that it will be dealing with parsed-character data within the tag. However, you will notice that some of the elements still have not been defined. Here is yet another update:

```
<?xml version="1.0"?>
<!DOCTYPE memberdata [
<!ELEMENT memberdata (member)+>
<!ELEMENT member (callsign, name, address, phone+,
                    class, email+?, arrl_member,
                    comment?)>
<!ELEMENT callsign (#PCDATA)>
<!ELEMENT name (prefix?, fname, minit?, lname,
                suffix?)>
<!ELEMENT prefix (#PCDATA)>
<!ELEMENT fname (#PCDATA)>
<!ELEMENT minit (#PCDATA)>
<!ELEMENT lname (#PCDATA)>
<!ELEMENT suffix (#PCDATA)>
<!ELEMENT address (street+, city, state, zipcode)>
<!ELEMENT street (#PCDATA)>
<!ELEMENT city (#PCDATA)>
<!ELEMENT state (#PCDATA)>
<!ELEMENT zipcode (#PCDATA)>
<!ELEMENT phone (areacode, localnum)>
<!ELEMENT areacode (#PCDATA)>
<!ELEMENT localnum (#PCDATA)>
<!ELEMENT class (#PCDATA)>
<!ELEMENT email (#PCDATA)>
<!ELEMENT arrl_member (#PCDATA)>
<!ELEMENT comment (#PCDATA)>
]>
```

All of the elements have now been defined. This file would be saved as 'sample.dtd'.

It is important to realize that there are more data types than #PCDATA available. Please look to the web sites in the bibliography to find more information on that topic.

Another exciting development is called XML-Data. Instead of using a standard DTD, XML-Data modifies the DTD somewhat, but it is still recognizable and useable as a DTD. XML-Data is a very new proposal, but promising. One of the problems with a standard DTD is that there is no way to show relationships between the elements defined. XML-Data is a proposal that allows for

relationships between data to be described in the DTD. This moves more towards the idea of using XML as a database. While this is possible, I do not advocate this use of XML unless it is used on a very small scale. XML tags add a large amount of overhead to data processing. While this may be a minimal consideration when dealing with the processing power of today's computer systems, it is still wasteful to use such an inefficient technique for storing large amounts of data. Remember, it would be very easy to store the data in a traditional database, and convert the needed records to XML dynamically. More information about XML-Data is available through the resources in the bibliography.

Step 3: Style Sheets.

If the XML document is to be displayed to screen through a browser or any other means, it is important that it be linked to a style sheet. The document as it stands is useful to non-human or data agents, but reading an XML document in its raw format would be a tedious process for a human user. However, the ability to read the database easily can be a major benefit of XML. It's very easy to correct database errors using only a text editor. I will not go into detail here about style sheets because there are many types available, but I will mention a few of the common style sheet languages. Even a brief explanation of each of the style sheet languages would require many pages, and it is not my intent to go into such depth here, as the prime focus of this document is XML.

Extensible Stylesheet Language (XSL) seems to be the best suited for displaying XML documents because it is a language all its own. XSL uses XML

syntax. XSL works very much like XML but is solely for the purpose of displaying data to screen. More information about XSL can be found through the resources in the bibliography, or by searching the web.

Cascading Style Sheets (CSS) can also be used to display XML documents. CSS is by no means a new development. Internet Explorer 4.0 has support for CSS, as do some other browsers. CSS is a good style sheet language, but XSL seems to be better suited to XML data.

It's important to note that almost every application that displays a document to screen uses a style sheet. Microsoft Word, Corel WordPerfect, and other word processors each have a proprietary style sheet. That's why it is necessary to import and export between the programs. This leads to another interesting use for XML. XML documents can also be directly converted to any document or spreadsheet format, if you have the correct parser.

Step 4: Parsers.

There are many utilities emerging on the web with which formatting an XML document is reduced to a simple task. If you'd rather not have to go through the bother of writing formatting scripts or programs yourself, it would be a worthwhile effort to search the web for XML parsers or formatting programs. Parsers check to see that an XML document is well formed and valid. Look to the resource section for more information about parsers.

IBM as such a program which allows the viewing of an XML document through a java applet, however at this time the applet appears to only work with the included demonstration XML. The applet would require extensive

modification to work with any other XML document. Search IBM's web site for more information on this parser.

The author of XML Applications has provided a very simple XML parser for demonstration, although it is not intended for use as an XML utility. It does give a good idea of how an XML document can be formatted and converted to HTML. The utility is available through links at <http://webdev.wrox.co.uk/books.1525/>.

Section 3: Resources

This section of the project is dedicated to resource information. There are several media types with which one can gain further knowledge of XML. The resources that I used for this project, and which I found most helpful, are listed here. Please note that here when I refer to W3, I'm indicating the World Wide Web Consortium.

This first book is where I found a great deal of basic information about XML and its used. It's worth while for the money. The book was about \$50 when I bought it.

Boumphrey, Frank, et al. XML Applications. Chicago: Wrox Press Inc., 1998.

This web site is the best way to start looking for XML info on the World Wide Web. It's basically the official XML index page. There's not much actual information here, but it gives a good collection of links that one can browse. This page has information about XML parsers and related applications.

Connolly, Dan. "Extensible Markup Language (XML)." W3, 1997. Online. Internet. 27 March 1999. Available <http://www.w3.org/XML/>

This page is a good summation of XML. As the name states, it's the XML FAQ.

Flynn, Peter et al. "Frequently Asked Questions about the Extensible Markup Language." W3: XML Special Interest Group, 6 October 1998. Online. Internet. 27 March 1999. Available <http://www.ucc.ie/xml/>

While the next web page is primarily dedicated to HTML, there is some good information here about how XML and HTML are becoming incorporated.

Raggett, Dave and Jacobs, Ian. "HTML Home Page." W3. Online. Internet. 27 March 1999. Available <http://www.w3.org/MarkUp>

This link provides more information about the XML-Data specification. It's quite a heavy document, but worth reading.

Layman, Andrew et al. "XML-Data." W3. Online. Internet. 27 March 1999. Available <http://www.w3.org/TR/1998/NOTE-XML-data/>

This page contains information about XML parsers and related programs. It's short, but a worth a good look.

Clark, James. "XML Resources." Online. Internet. 27 March 1999. Available <http://www.jclark.com/xml/>

For more information about style sheets, this is a good link to look at. It contains information about XSL, CSS and DSSSL

Lie, Håkon Wium. "Web Style Sheets." W3. Online. Internet. 27 March 1999. Available <http://www.w3.org/Style/>

Information about XHTML is available through the following link. XHTML is an exciting new prospect.

Pemberton, Steven et al. "XHTML 1.0: The Extensible HyperText Markup Language." W3. Online. Internet. 27 March 1999. Available <http://www.w3.org/TR/WD-html-in-xml/>

This next page is a source for new happenings in SGML and XML.

Cover, Robin. "SGML and HTML News." Online. Internet. 27 March 1999. Available

<http://www.oasis-open.org/cover/sgmlnew.html>

Conclusion

By now, the reader should have a good idea of what XML is and what its uses are. There is a lot more information available on the World Wide Web. If you're interested, start by looking at the sites listed in the resource section.

XML should become part of the World Wide Web landscape by the end of the year. Microsoft has released IE 5.0, and Netscape should soon release the 5.0 version of Communicator. IE 5.0 already contains some XML capabilities, and Communicator 5.0 should also. However, we'll just have to wait and see how Microsoft and Netscape decide to deal with XML implementation. I think that it will be a few years before XML gains wide use. However, just as Linux was once considered a hacker's operating system, and is now considered suitable for business use, so shall XML be adopted on the Web. Happy surfing!